KnowOps AI

This project is a small web app that answers DevOps/IT support questions by reading a set of "runbook" documents (how-to guides for fixing servers, outages, etc.) and also has a built-in "agent" that can look things up and manage support tickets on its own. Think of the whole repository (the project folder) as a house: each top-level folder is a room with one job, and the files inside are the furniture in that room. Below is a tour, room by room, in plain language — no coding background needed.

How to read this guide

Each section below names one folder or file exactly as it appears in the project, in **`monospace text like this`**, followed by a short, plain-English explanation of what lives there and why it matters.

The 30-second overview

Folder / File	What it is, in one line
`app/`	The actual brain of the app — all the Python code that answers questions
`api/`	A tiny bridge file so the app can run on Vercel (the hosting service)
`scripts/`	One-off helper programs (loading documents in, generating reports)
`data/docs/`	The support documents (runbooks) the AI reads answers from
`static/`	The website you see in your browser (HTML/CSS/JS)
`tests/`	Automated checks that make sure the code still works correctly
`n8n/`	A saved automation workflow for a separate tool called n8n
`AGENTS.md / CLAUDE.md`	Instructions for AI coding assistants working on this project
`DECISION.md`	A diary of why certain technical choices were made
`FLOW.md`	A map of "what calls what" inside the code
`README.md`	The welcome page — what this project is and how to start it
`requirements.txt`	The shopping list of Python add-ons the app needs
`.env.example`	A template for secret settings (passwords, API keys)
`vercel.json`	Settings telling the Vercel hosting service how to run the app

The rest of this guide goes folder by folder with a bit more detail and the key files inside each one.

The folders, one by one

`app/`

The engine room — all the Python code that makes decisions.

- **main.py** — the front door; every web request (like "ask a question" or "create a ticket") arrives here first and gets routed to the right piece of code.

- **auth.py** — the bouncer; checks logins and hands out a digital "pass" (called a JWT) so the app knows who you are on later requests.
- **rag.py** — the librarian; searches the stored documents for the paragraphs most relevant to your question.
- **llm.py** — the translator to the AI model (Google Gemini); sends it the question plus the relevant paragraphs and gets back a written answer.
- **agent.py** — the assistant that can take multiple steps on its own (search, check a ticket, update a ticket) to resolve a request, not just answer one question.
- **crm.py** — the ticket system; creating, listing and updating support tickets.
- **db.py** / **config.py** — plumbing: how the app connects to its database and reads its settings.
- **schemas.py** — the rulebook for what a valid request/response looks like (e.g., a question must be text, not a number).

`api/`

A one-file adapter so the app can live on Vercel's hosting.

- **index.py** — just points Vercel at the real app in `app/main.py`. You will basically never need to open this file.

`scripts/`

Programs you run by hand occasionally — not part of the live website.

- **ingest.py** — reads every document in `data/docs/`, breaks it into bite-sized chunks, and loads it into the database so the AI can search it.
- **init_db.sql** — the one-time setup script that creates the database tables.
- **generate_phase_*_pdf.py** — scripts that generate the project's PDF status reports (this very PDF was made the same way).

`data/docs/`

The knowledge base — the actual support documents the AI learns from.

- Plain Markdown (`.md`) files such as `disaster_recovery_runbook.md` or `kubernetes_operations.md` — real troubleshooting guides.
- Nothing clever here: it's just text files. The AI never "makes up" answers outside of what's written in these documents.

`static/`

The part of the app you actually see and click on in a browser.

- **index.html** — the page's structure (the chat box, the tickets tab, etc.).
- **css/app.css** — the page's look (colors, spacing, fonts).
- **js/app.js** — the page's behavior (what happens when you click "Ask" or "Log in").

`tests/`

Automated "does it still work?" checks, run with one command (pytest).

- Each `test_*.py` file checks one part of the app (login, tickets, the agent, the search). Running them after a change catches mistakes before real users do.

`n8n/`

One saved workflow for n8n, a separate no-code automation tool used for scheduled tasks.

- **incident_triage_workflow.json** — the exported workflow definition; opened inside the n8n app, not part of the website itself.

Key files at the top level

`AGENTS.md (read via CLAUDE.md)`

The single "source of truth" for project status and the rules AI coding assistants must follow.

- Lists every phase of the build, who owns it, and whether it's done.
- Also defines the house rules (log decisions, keep the flow docs honest, quiz before accepting a change).

`README.md`

The welcome page anyone sees first on GitHub.

- What the project does, the tech stack, and the exact commands to get it running locally.

`DECISION.md / FLOW.md`

The project's memory.

- **DECISION.md** — a running log of why choices were made (e.g., why this database, why this chunk size).
- **FLOW.md** — a map of which function calls which, useful when tracing a bug.

`requirements.txt`

The exact list of Python add-on packages (like FastAPI, the web framework) the project needs installed.

`.env.example`

A template listing every secret setting the app needs (database link, API keys, passwords) without the real values — so nobody accidentally commits actual secrets to GitHub.

`vercel.json`

Configuration for Vercel, the service that hosts the live website — tells it which file to run and when to trigger scheduled jobs.

Quick tip: if you just want to see the project running, you don't need to touch most of this — install the requirements, copy `.env.example` to `.env` and fill in real values, then run `uvicorn app.main:app --reload` and open the site in a browser.